

Day 3 - Data Structures: Arrays and Objects, Destructuring, Spread/Rest

Day 3 — Data Structures: Arrays and Objects, Destructuring, Spread/Rest

Objectives

- Get fluent with arrays: indexing, the core methods, and the crucial `map/filter/reduce` trio
- Get fluent with objects: JavaScript's key-value collection, roughly equivalent to Python's dict
- Understand destructuring — a compact way to pull values out of arrays/objects into variables
- Understand the spread (`...`) and rest (`...`) syntax — the same three dots, doing related but different jobs depending on context

Arrays — ordered collections

An array is written with square brackets, values separated by commas — visually identical to a Python list:

```
const fruits = ["apple", "banana", "cherry"];
```

Just like Python, items are indexed starting at 0:

```
fruits[0]      // "apple"
fruits[2]      // "cherry"
fruits.length  // 3 -- a PROPERTY (no parentheses), not a function call, unlike Python's len(fruits)
fruits[10]     // undefined -- NOT an error! Reading past the end just gives you undefined
```

This last point is a genuine, important difference from Python worth noting now: reading an out-of-range index in JavaScript does not raise an error the way Python's `IndexError` does — it silently gives you `undefined`. This means a bug from an out-of-range index can go unnoticed for longer in JavaScript, since nothing crashes immediately — you'll just get confusing `undefined` values showing up further along in your program. Get in the habit of checking `array.length` when the range is uncertain.

Core array methods

```
const nums = [3, 1, 2];

nums.push(4);           // adds to the END, returns the new length -- [3, 1, 2, 4]
nums.pop();             // removes AND returns the last item -- nums is now [3, 1, 2]
nums.unshift(0);        // adds to the START -- [0, 3, 1, 2]
nums.shift();           // removes AND returns the first item -- [3, 1, 2]

nums.includes(1);        // true -- does the array contain this value?
nums.indexOf(1);         // 1 -- the INDEX where 1 is found, or -1 if not found
nums.slice(0, 2);        // [3, 1] -- like Python slicing: start included, end excluded
nums.join(", ");        // "3, 1, 2" -- combine into one string, like Python's ' '.join(...)
[...nums].sort((a, b) => a - b); // sorted COPY -- see the spread section below for why
```

`.sort()` on its own has a famous gotcha: by default it sorts based on treating everything as TEXT, not numbers — `[10, 2, 1].sort()` gives you `[1, 10, 2]`, not `[1, 2, 10]`, because it compares the strings "10", "2", "1" alphabetically. To sort numbers correctly, you must pass a **comparator function**: `nums.sort((a, b) => a - b)` (ascending) — this is worth memorizing as a pattern now, even before Day 4 fully explains that `(a, b) => a - b` syntax (a small anonymous function, called an "arrow function").

``map`, `filter`, and `reduce`` — the three most important array methods you'll use, constantly, for the rest of your career

These three methods are how idiomatic, modern JavaScript transforms and processes collections of data — you will see and use them daily in any real JavaScript job. Each one takes a small function as an argument (this is a preview of Day 4's "functions are values" idea) describing what to do to each item.

`.map()` — transform every item into something new, producing a brand new array of the same length:

```
const numbers = [1, 2, 3, 4];
const doubled = numbers.map((n) => n * 2);
console.log(doubled); // [2, 4, 6, 8]
```

Read `(n) => n * 2` as: "for each item, call it `n`, and produce `n * 2`." This is the direct equivalent of Python's list comprehension `[n * 2 for n in numbers]`.

`.filter()` — keep only the items that pass a test, producing a new (possibly shorter) array:

```
const evens = numbers.filter((n) => n % 2 === 0);
console.log(evens); // [2, 4]
```

Equivalent to Python's `[n for n in numbers if n % 2 == 0]`.

`.reduce()` — combine every item down into a single result (a sum, a maximum, a single merged object — anything), by repeatedly applying a function that takes the "running total so far" and the "current item":

```
const total = numbers.reduce((sum, n) => sum + n, 0);
console.log(total); // 10
```

Read this carefully: `(sum, n) => sum + n` is a function of two things — `sum` (the running total so far) and `n` (the current item) — and the `0` at the very end is the **starting value** for `sum`, before the first item is even processed. `.reduce()` is genuinely the trickiest of the three to get comfortable with — don't worry if it takes a few tries in the exercises below to feel natural; it's worth the effort, since it's extremely common in real code (it's how you compute totals, group data, flatten nested arrays, and much more).

You'll use all three of these constantly from here on, often chained together:

```
numbers.filter(...).map(...).reduce(...).
```

Objects — key-value collections

An **object** is JavaScript's roughly-equivalent-to-Python's-dict — a collection of named properties, written with curly braces:

```
const person = {
  name: "Ana",
  age: 30,
  isActive: true,
};
```

```

person.name          // "Ana" -- "dot notation," the more common way to access a property
person["age"]        // 30 -- "bracket notation," required when the key is stored in a variable

person.email = "ana@example.com"; // adding a brand new property is just an ordinary assignment
person.age = 31;                // updating an existing property, same syntax
delete person.isActive;         // removes a property entirely

```

Use dot notation (`person.name`) whenever the key is a fixed, known name written directly in your code — it's shorter and clearer. Use bracket notation (`person[someVariable]`) when the key itself is stored in a variable, or when the key contains characters that wouldn't be valid as a plain identifier (like a space or a number at the start):

```

const key = "age";
console.log(person[key]); // 30 -- the variable `key` holds the STRING "age", used to look up the value

```

Looping over an object

```

for (const key in person) {
  console.log(key, person[key]);
}

Object.keys(person); // ["name", "age", "email"] -- an ARRAY of just the keys
Object.values(person); // ["Ana", 31, "ana@example.com"] -- an ARRAY of just the values
Object.entries(person); // [["name", "Ana"], ["age", 31], ...] -- an ARRAY of [key, value] pairs

```

`Object.entries()` combined with `for...of` is a common, clean way to get both key and value together, closely mirroring Python's `dict.items()`:

```

for (const [key, value] of Object.entries(person)) {
  console.log(key, value);
}

```

Checking whether a key exists

```

"name" in person // true -- checks whether the property exists at all, even if its value is undefined
person.name !== undefined // a common but slightly imperfect alternative -- fails if the value is null
person.hasOwnProperty("name") // true -- another common, explicit way to check

```

Destructuring — pulling values out into variables in one step

Destructuring lets you unpack values from an array or object directly into separate variables, in one concise line, instead of accessing each one individually:

```

// Array destructuring -- position-based
const coordinates = [10, 20];
const [x, y] = coordinates;
console.log(x, y); // 10 20

// Object destructuring -- NAME-based (must match the property names exactly)
const person = { name: "Ana", age: 30 };
const { name, age } = person;
console.log(name, age); // Ana 30

// You can rename while destructuring an object:

```

```
const { name: personName } = person;
console.log(personName);    // Ana

// And provide a default, used only if that property is missing/undefined:
const { country = "Unknown" } = person;
console.log(country);    // "Unknown" -- person had no `country` property at all
```

You'll see object destructuring constantly in function parameters starting Day 4 — it's an extremely common, idiomatic JavaScript pattern.

Spread (...) and rest (...) — the same three dots, two different jobs

This is a common point of confusion for beginners: `...` looks identical in both uses, but does the *opposite* job depending on where it appears.

Spread — expands an existing array or object out into its individual elements, most often used to build a new array/object or to copy one:

```
const nums1 = [1, 2, 3];
const nums2 = [...nums1, 4, 5];    // [1, 2, 3, 4, 5] -- spreads nums1's items out, then adds more
const numsCopy = [...nums1];        // a genuine, independent COPY of nums1 -- NOT the same array

const person = { name: "Ana", age: 30 };
const updatedPerson = { ...person, age: 31 };    // { name: "Ana", age: 31 } -- copy person, then override
```

That last example is an extremely common real-world pattern: "make a new object just like this one, but with one field changed" — spreading first, then listing overrides afterward (later properties win over earlier ones with the same name).

Rest — the opposite operation: **gathers** multiple individual values back together into a single array, most often seen in function parameters (full treatment Day 4) or destructuring:

```
const [first, ...rest] = [1, 2, 3, 4];
console.log(first);    // 1
console.log(rest);    // [2, 3, 4] -- everything ELSE gets gathered into this array
```

A rough rule for telling them apart when you see `...` in someone else's code: if it's spreading an **existing** collection out (usually on the right-hand side of an assignment, or inside a new array/object literal), it's spread. If it's gathering loose values **together** into one new array (usually on the left-hand side, in a destructuring pattern, or as a function parameter), it's rest.

Exercises

Open `exercises.js`, fill in each `// TODO`, and run `node exercises.js`.